fossenCD Design Document

fossenCD is a self-hostable collaborative Typst editor built on teamtype, a peer-to-peer CRDT editing protocol over iroh. It bridges native peers (people editing files locally with their own editor + the teamtype daemon) and web peers (people editing in the browser via a WebAssembly peer), with a Go backend providing accounts, projects, membership, and supervised hosting, and a Vue frontend for the web editing experience. This document describes the architecture, the identity and trust model, project hosting, and the design for signed comments and suggestions.

Contents

1. Overview

fossenCD (Free and Open-Source Software, Editor-Native Collaborative Documents) is a self-hostable collaborative document editor for Typst. It is built on teamtype, a peer-to-peer collaborative editing protocol (with an included Rust implementation) that synchronizes a shared Automerge CRDT document over iroh (for relaying) and Magic Wormhole (for out-of-band join codes). The defining property of teamtype is that it is fully peer-to-peer and local-first: the document is stored in a local directory, and a teamtype daemon synchronizes it with other peers over the network, but there is no required central server or service, nor do peers have to be online at the same time.

fossenCD builds on this with a Go backend that provides project hosting with persistent teamtype peers (for users to connect to), a Vue frontend for web editing, and an identity and access control layer, which altogether allows both local Teamtype peers and web peers to collaborate on the same document.

In other words, fossenCD is to teamtype roughly what Gitea is to git. Just as Gitea provides a self-hostable higher-level identity layer over git's distributed version control, fossenCD provides accounts, access control, a web editing experience, and an always-online peer on top of teamtype's peer-to-peer CRDT synchronization protocol, scoped deliberately to Typst the way Overleaf scopes itself to LaTeX.

1.1. Editing modes

fossenCD supports two first-class ways to edit the same document, concurrently:

- **Local:** The user runs the teamtype daemon against a directory on their own machine and edits the files with any editor (e.g. Neovim, Emacs, VS Code). They are a durable peer with a persistent identity and full offline history.
- **Web:** The user opens the project in a browser. A WebAssembly teamtype peer runs in the page, connects to the project's host, and synchronizes live. This peer is ephemeral and holds no persistent identity and no local history beyond the session.

Both modes speak the same protocol and edit the same CRDT, so a local user in Neovim and a web user in the browser collaborate on the same document in real time.

1.2. Goals

- Self-hostable end to end; no required dependency on any cloud service we run.
- Local and web peers as equal collaborators on one document.
- Real-time collaboration with offline-capable local replicas.
- Identity and access control that fossenCD owns, layered cleanly over teamtype's transport and CRDT identities.
- Signed, verifiable comments and suggestions that work locally without contacting a server.
- The protocol, peer client, and frontend are all open source (AGPL-3.0).

1.3. Non-goals

- Not a general-purpose document platform. Currently, this project only aims to support Typst.
- Not a centralized SaaS.
- Not a replacement for teamtype; fossenCD depends on it and tracks upstream.

1.4. Design principles

The synced document is the only thing every peer sees Local Teamtype replicas speak only teamtype's P2P protocol, and so the fossenCD service appears invisible. Anything *every* collaborator must observe therefore has to live inside the synchronized CRDT.

Identity is layered Transport identity (iroh via teamtype), authorship identity (Automerge via teamtype), and account identity (fossenCD) are distinct, and bound to each other explicitly where needed rather than assumed equal.

The system provides convenience and is not a single point of truth An example is that secret addresses and join codes needed for peers to connect can be derived deterministically (by deriving public key from `.teamtype/key` for iroh nodes and generating a passphrase required by each peer, which is exchanged through Magic Wormhole). In fossenCD, the system prefers to not ask the daemon and instead derives the values by itself, but the core derivation must remain the same as how the teamtype daemon would derive them, and the backend should *not* own the keys.

Follow git's trust model Local-first signing and verification, with the server as an auto-verifier (plus additional keyservers).

1.5. Tech stack

- **Synchronization:** Teamtype (Automerge CRDT, iroh transport, magic-wormhole for join codes)
- **Web peer:** teamtype-wasm, a WebAssembly build of a Teamtype peer (`@sandvichxyz/teamtype-wasm`)
- **Backend:** Go: chi + huma for the HTTP/OpenAPI layer, GORM over SQLite for persistence
- **Frontend:** Vue 3 + TypeScript + Pinia, with a generated OpenAPI client
- **Reverse Proxy:** nginx

2. Identity and trust

fossenCD juggles three distinct identities. They are kept separate and bound together only where a binding is actually meaningful.

2.1. Identity Layers

2.1.1. Managed by Teamtype

- **iroh NodeId (transport):** An ed25519 public key. It is the cryptographic identity of a peer's network endpoint. Connections are authenticated against it by QUIC/TLS, and access to a project is additionally gated by a passphrase exchanged in the handshake. The secret address a peer connects by is `<host-node-id>#<passphrase>`.
- **Automerge ActorId (authorship):** A per-peer identifier Automerge uses to attribute each change in the CRDT. It is what distinguishes "who made this edit" at the document level, independent of the transport. A single peer's cursors are keyed as `<actor-id>-<editor-id>`.

2.1.2. Managed by fossenCD

- **fossenCD account:** A durable user in the backend's database. The account identity owns projects, holds membership, and public keys. This is the only layer that corresponds to a *person* across sessions and projects.

2.2. Key Material

A Teamtype instance stores the private key in `.teamtype/key`, which the public key can be derived from. The public key is not stored anywhere but is used for registering with iroh relays and identifying iroh nodes.

fossenCD's backend reads and writes that same key file, so it can derive a project's secret address (and rotate it by changing the file) without the daemon running. Every local Teamtype peer, whether it ran `share` or `join`, has its own `.teamtype/key`, hence its own NodeId.

2.2.1. Web Peer

The browser peer is different: it has no filesystem to persist a key, and it does not store one. Currently, it generates a fresh keypair in memory per session, although this optionally could be stored in `localStorage` in the future. This means:

- Every page load is a brand-new peer.
- Therefore authorship that must survive a session (such as comments and suggestions) cannot rely on the Automerge ActorId or iroh NodeId. It must be anchored to the fossenCD account, via a signature (see Section 4).

2.3. Binding the Layers

The only binding that carries weight is *account to signing key*. A comment's authorship is established by a signature whose key the backend (acting as a keyserver) associates with a fossenCD account.

3. Project Hosting

For any Teamtype peer to join a share (in particular, for a fossenCD web user to open a project), *some* peer must be online to synchronize with. A hosted fossenCD instance fills that role by running an instance of teamtype per project to accept connections and synchronize changes at any time.

3.1. Addresses and join codes

The backend derives a project's secret address directly from its `.teamtype/key` (node id from the seed, passphrase from the key's second half), so addresses can be served over the API without involving the daemon. A join code is a fresh random magic-wormhole code. The backend mints it in-process and hands the key-derived secret address over the wormhole channel when a peer claims it, rather than scraping the code from daemon output.

3.2. Access control and revocation

Project access is gated by a passphrase. In vanilla Teamtype, that passphrase *is* the second half of `.teamtype/key` (see Section 2). So the secret address `nodeid#passphrase` is one capability shared by everyone, and the only way to revoke a member is to rotate the key (generate a new `.teamtype/key`), which yields a new NodeId, address, and passphrase, then redistribute the new address to the members who remain on request. This breaks for local users who join through `teamtype join` with a configured secret address without passing in a join code. There are two ways to be able to solve this:

1. Fork `teamtype` to make a persistent server version, and give each peer a unique passphrase (which must be stored, ideally in `.teamtype` to persist across restarts)
2. Expose a convenient way (like a `curl | sh` script) to retrieve the current secret address; currently we expose an authenticated endpoint for retrieving the secret address, and this just takes the response and writes it to the Teamtype config

Option 2 is currently the easiest way to make the revocation UX acceptable. In the future, a dedicated Teamtype server would be beneficial in the long run to provide further granular access control and other features.

4. Comments and Suggestions

Comments and suggestions must reach every collaborator, including local Teamtype replicas that never contact the backend. They must live inside the synchronized document. The model below is similar to how git commits can be signed, but each entity has its own way of verifying and trusting parties.

4.1. Persistence

Comments are stored as files in a synchronized path in the working tree (e.g. `.fossencd/`). These files would ride the CRDT, so that every peer (including local Teamtype users) receives them for free, offline, with no separate transport. The cost is that local users see comment files in their tree, but this is acceptable and arguably git-like.

4.2. Immutability

A signed object cannot be edited in place without breaking its signature, so comments are *immutable*. This is turned into an advantage: the comment store is an append-only set of immutable signed objects, exactly like git objects.

- Editing a comment is a new signed object that supersedes the prior one.
- Resolving or threading is new objects referencing parents.
- The CRDT therefore only ever *grows*. No two peers concurrently mutate the same comment blob, so the conflict problem of mutable comment files disappears entirely.

Each object's signature covers a canonical serialization of: the author identity, the content, the anchor, the file path, a timestamp, any parent reference, and an optional staleness hash (the hash of the text contained in the comment's selected range). Any tampering or post-hoc edit breaks the signature.

4.2.1. Preventing Tampering

The signature is a non-repudiation proof of authorship but does not prevent deletion or replacement at the filesystem layer by a malicious party with write access to the repository. For most use cases, a shared document implies some sort of mutual trust, so this is not a problem.

But to solve this problem, because automerge is inherently append-only, any comment object that is deleted or replaced by a malicious party would still exist in the CRDT history. The caveat is that recovery is a manual search through the history, around a $O(n)$ time complexity for n CRDT operations.

A question for the future is how to surface this in the UI? One possibility is showing a warning that "this exists in the backup but is missing from your view; restore?" Without that UX, the append-only guarantee serves as only a safety net that nobody looks at, but it is still a strong guarantee that comment data is not truly lost.

4.3. Anchoring

A comment points at a range in a document, but every keystroke shifts byte offsets. Anchors therefore do not store offsets; they store an Automerge cursor (a stable, edit-tracking reference) into the target file's text. Because teamtype mirrors the whole directory as a single Automerge document, a comment object in `.fossencd/` can reference a cursor in `main.typ` and follow its text through edits.

The signed payload also includes a hash of the anchored substring at creation time. When the underlying text later changes, the live text no longer matches the hash, and the comment can be rendered *outdated* — the equivalent of GitHub graying out a review comment whose lines have moved.

4.4. Suggestions

A suggestion extends a comment: it additionally carries a proposed replacement for its anchored range and an accept/reject state. Accepting a suggestion applies the edit to the document. It reuses the same anchoring, signing, and immutability machinery, plus the patch payload and resolution state.

4.5. Signing

Comment authorship rides the signature, decoupling it from the throwaway transport and CRDT identities. There are two signing paths, mirroring git and GitHub:

- **Local signing:** A local Teamtype user signs the object with their own durable signing key, similarly to `git commit -S`. This works fully offline without any server involved. The friction of holding and managing a key is accepted, by design. Note this is a *user-level* signing key, not the per-directory `.teamtype/key` transport key.
- **Server-assisted signing:** A web user can not use or upload a private key to use for signing, so they contact the fossenCD service to sign off for them, similarly to GitHub's `web-flow` user signing off the user's web commits.

4.6. Verification

Verification is local-first, like git, which never auto-verifies:

- **Local verification:** A local client verifies signatures itself against a keyring assembled from the keyservers it is configured to trust.
- **Server-assisted verification:** The backend can auto-verify and surface a badge in the web UI.

From fossenCD's point of view, a comment is considered verified if *at least one* public key, from *any* of the configured keyservers, validates its signature (OR across the trusted keyserver union). The set of keyservers is configured by the instance administrator. This is useful if an organization might run its own keyserver for its members, or if a public instance administrator chooses to trust a broad public keyserver plus its own.

4.6.1. Keyserver Federation

The instance itself is a keyserver, similar to GitHub: users upload their own public keys, and the server serves them for verification. Two federation features extend this:

- The instance administrator can add other keyservers as lookup sources, so the instance can resolve and trust keys it does not itself hold.
- Users can upload their own public keys to their account.